SRE CASE STUDY

AZURE ARC HYBRID CLOUD LAB

Parts 1-3: Kubernetes, Azure Arc, Terraform, CI/CD, Ingress TLS, HPA, NetworkPolicy, RBAC

Author: Bartosz Suszko

Company: IT Solutions Bartosz Suszko, Olsztyn, Poland

Date: May 2026 (Parts 1-3)

GitHub: github.com/buth11/sre-homelab-azure-arc

Stack: Kubernetes | Azure Arc | Terraform | KQL | Azure DevOps | Traefik | HPA | NetworkPolicy | RBAC

Table of Contents

Chapter 1: Context and Architecture

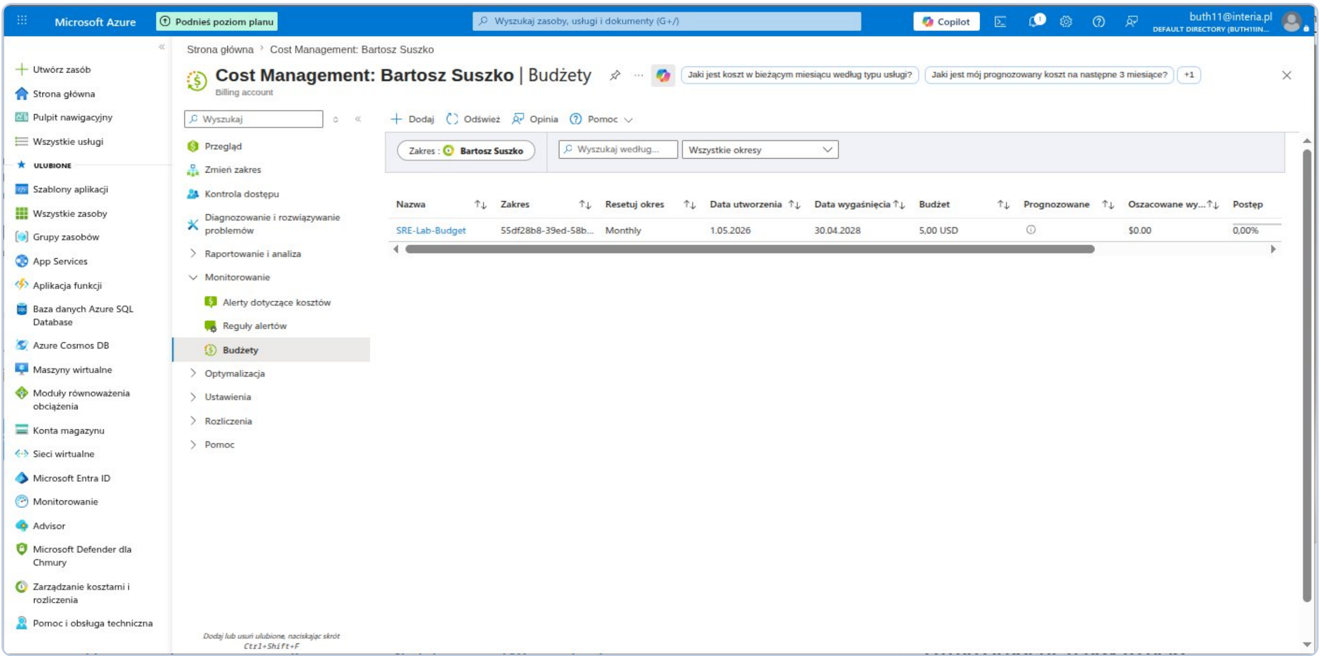
This project demonstrates building a production-grade hybrid environment on GMKtec EVO-X2 (128 GB RAM) connecting Kubernetes with Microsoft Azure through Azure Arc. Three parts of hands-on practice with documentation of every step and 10 documented incidents.

Layer	Technology	Purpose
Kubernetes	K3s v1.35.4+k3s1	3-node cluster
Ingress	Traefik + TLS	HTTPS domain routing
Autoscaling	HPA (autoscaling/v2)	CPU-based scaling 2-8 replicas
Network Security	NetworkPolicy	Intra-cluster firewall
Access Control	RBAC + ServiceAccount	Least privilege
Cloud Bridge	Azure Arc	Hybrid control plane
Monitoring	Azure Monitor + Prometheus	Observability
IaC	Terraform + Remote State	Infrastructure as Code
CI/CD	Azure DevOps	Pipeline with approval gate

Chapter 2: On-Premise Infrastructure

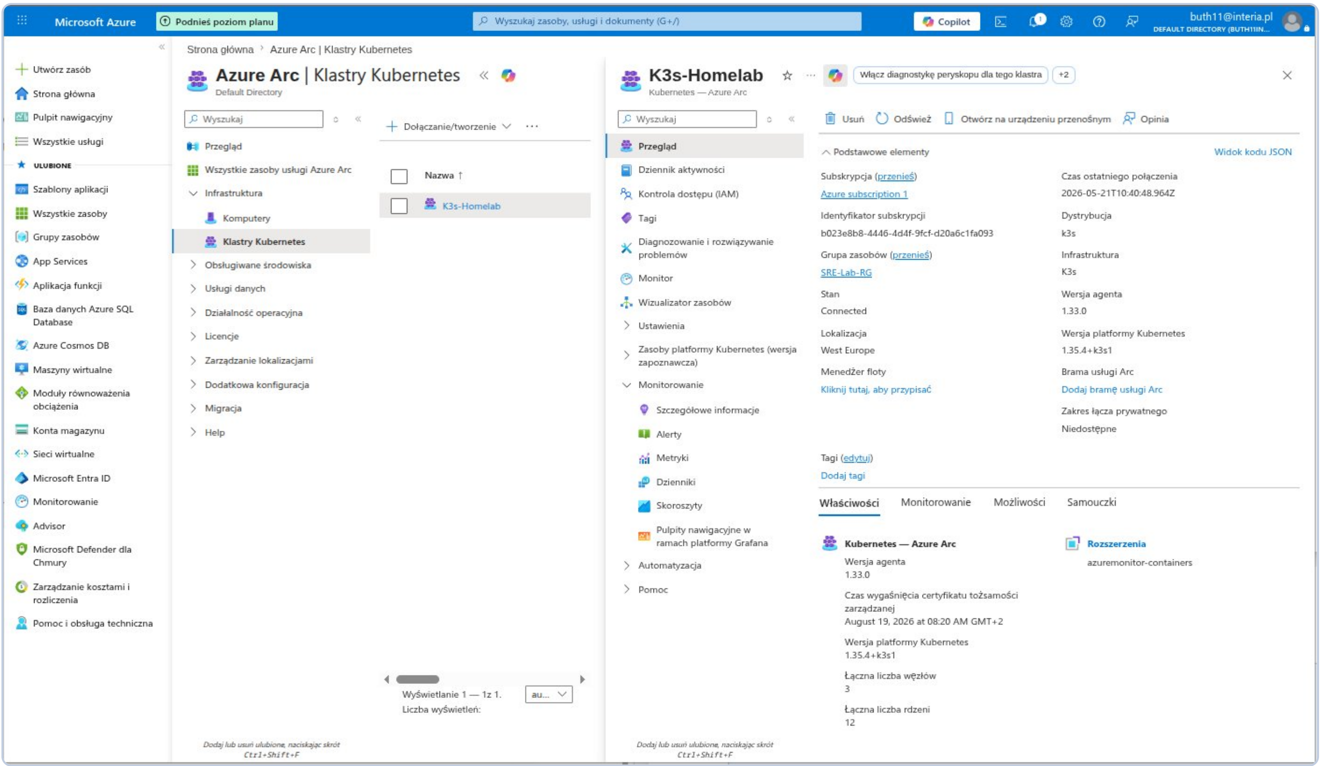
Node	IP	CPU	RAM
k3s-master (Control Plane)	192.168.122.10	4 vCPU	8 GB
k3s-worker1	192.168.122.11	4 vCPU	16 GB
k3s-worker2	192.168.122.12	4 vCPU	16 GB

Chapter 3: FinOps



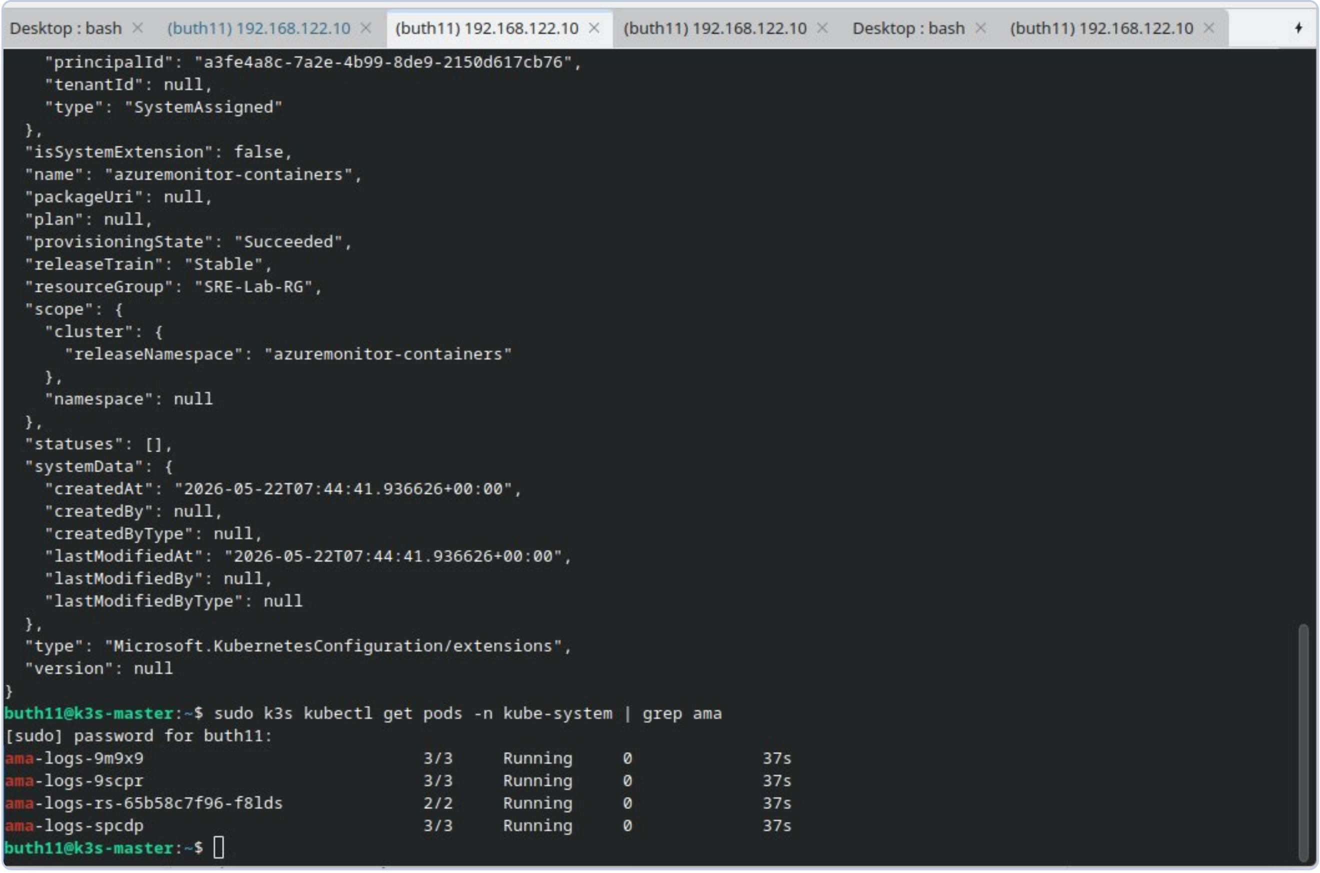
Screenshot 1: Azure Cost Management — 5 USD/month budget, cost: 0.00 USD.

Chapter 4: Azure Arc

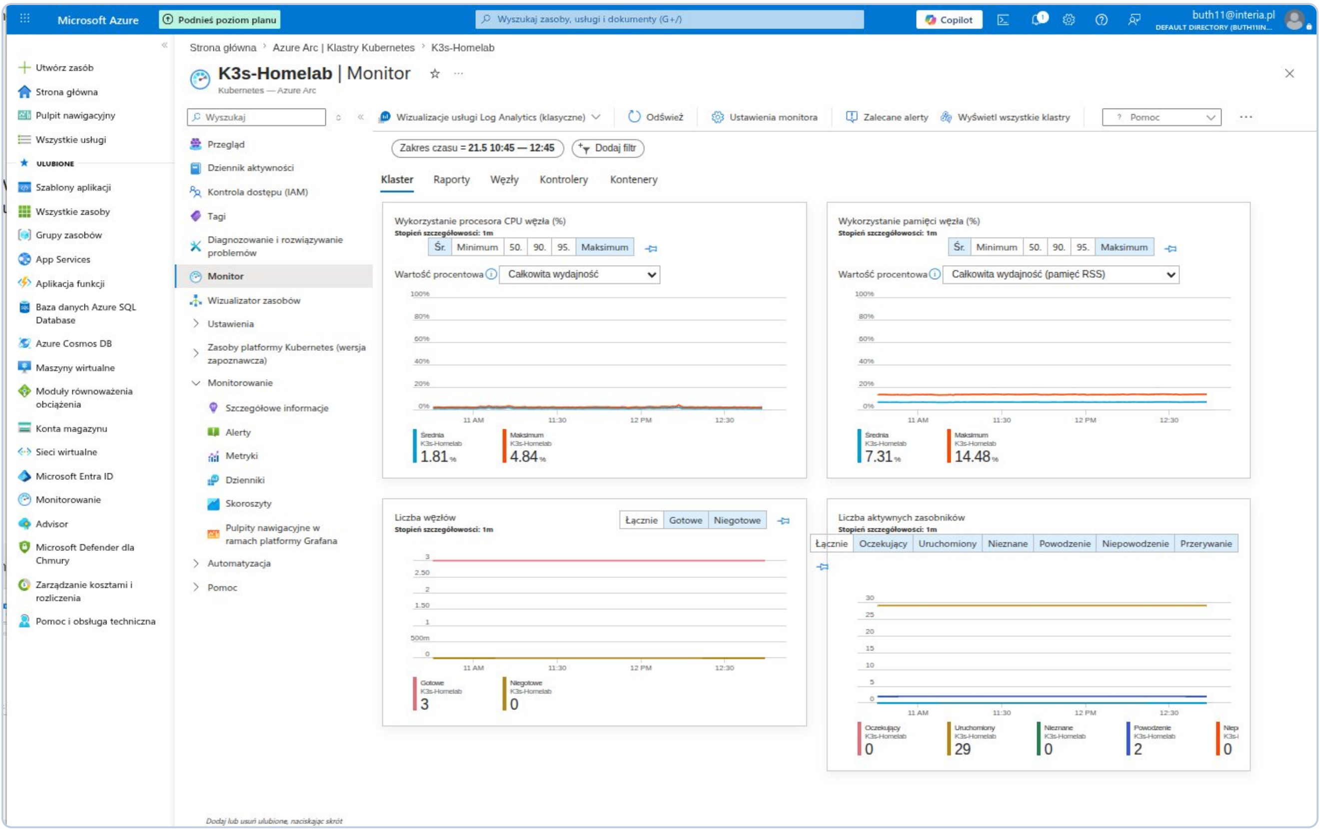


Screenshot 2: Azure Arc — K3s-Homelab Connected, v1.35.4+k3s1, 3 nodes, 12 cores.

Chapter 5: Observability

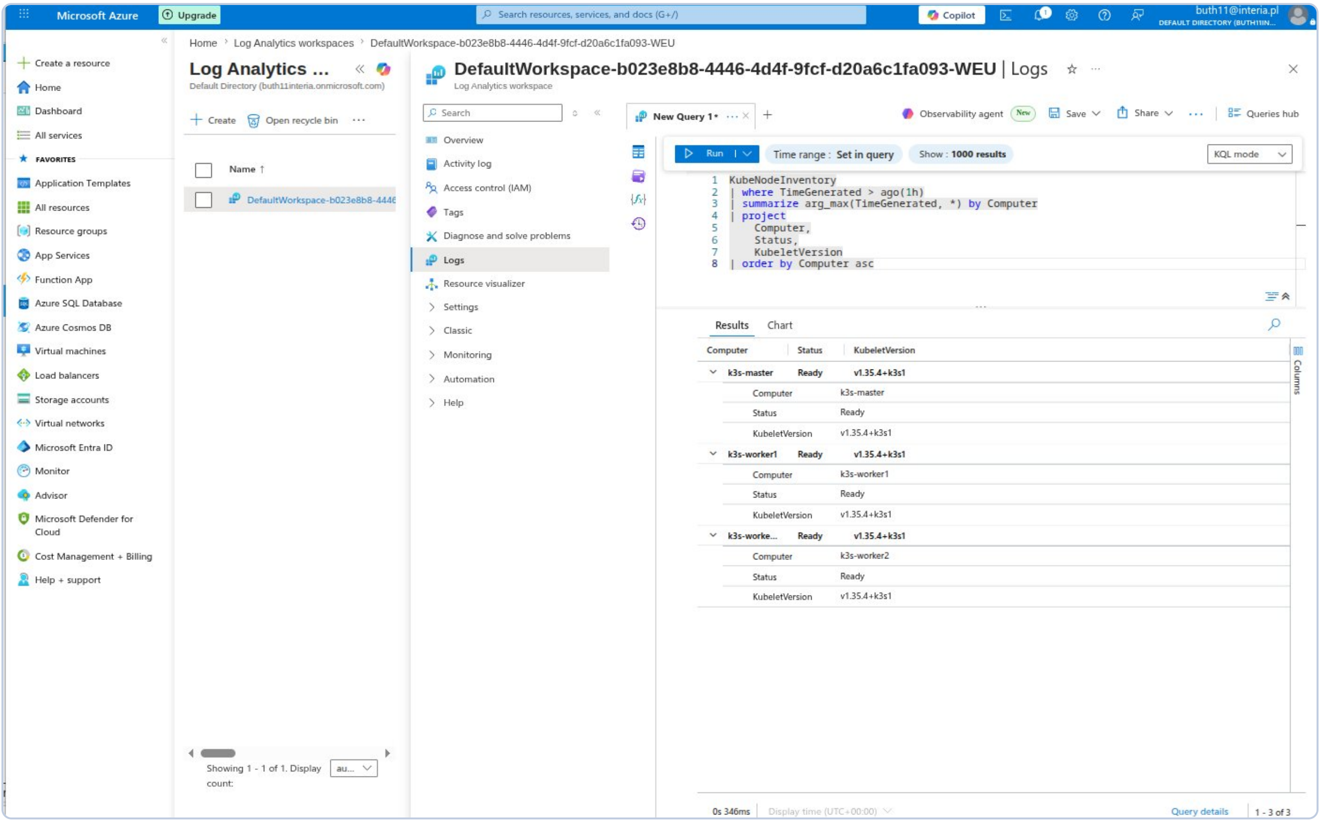


Screenshot 3: AMA pods Running — 4 agents, 0 restarts.

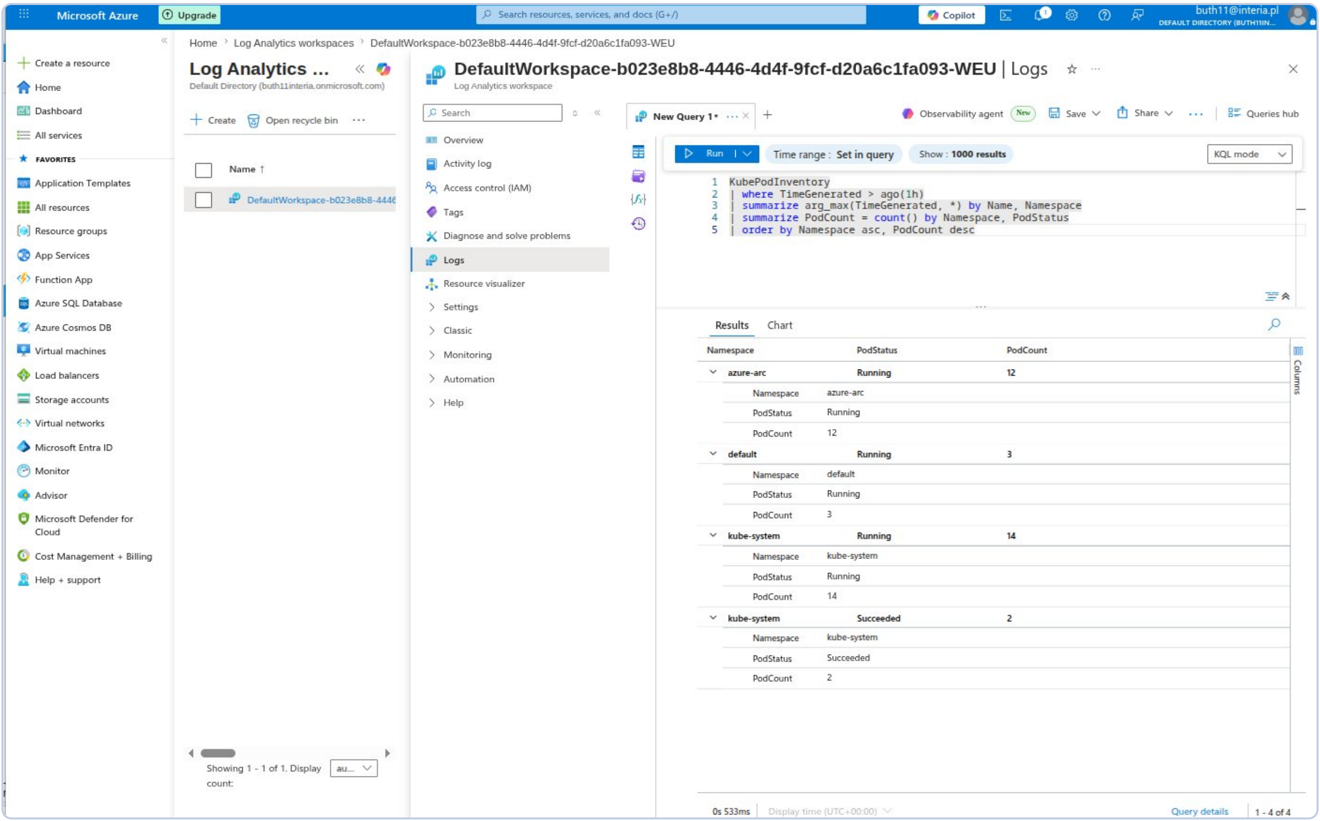


Screenshot 4: Azure Monitor — CPU avg 1.58%, RAM avg 6.27%, 3 nodes Ready, 29 pods.

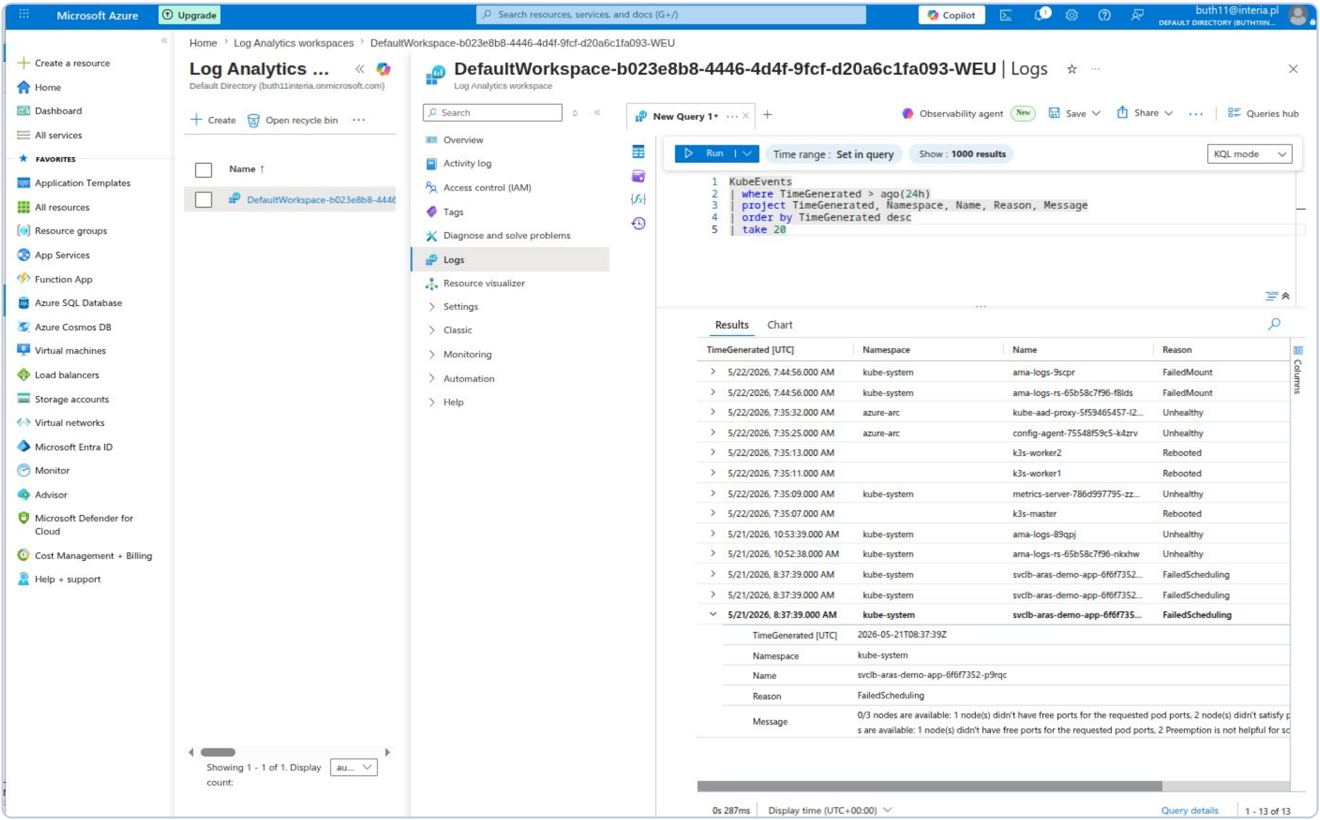
Chapter 6: KQL



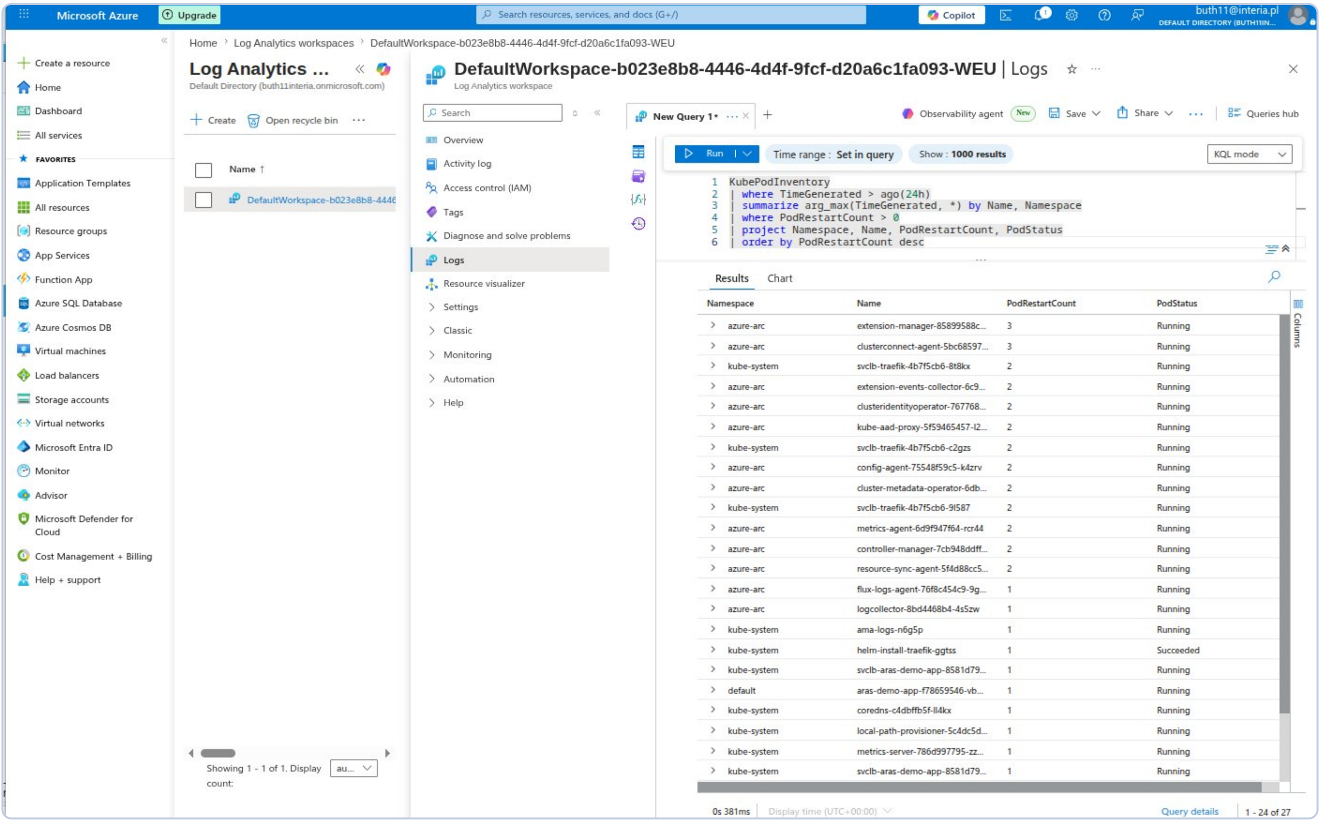
KQL-1: 3 nodes Ready, v1.35.4+k3s1.



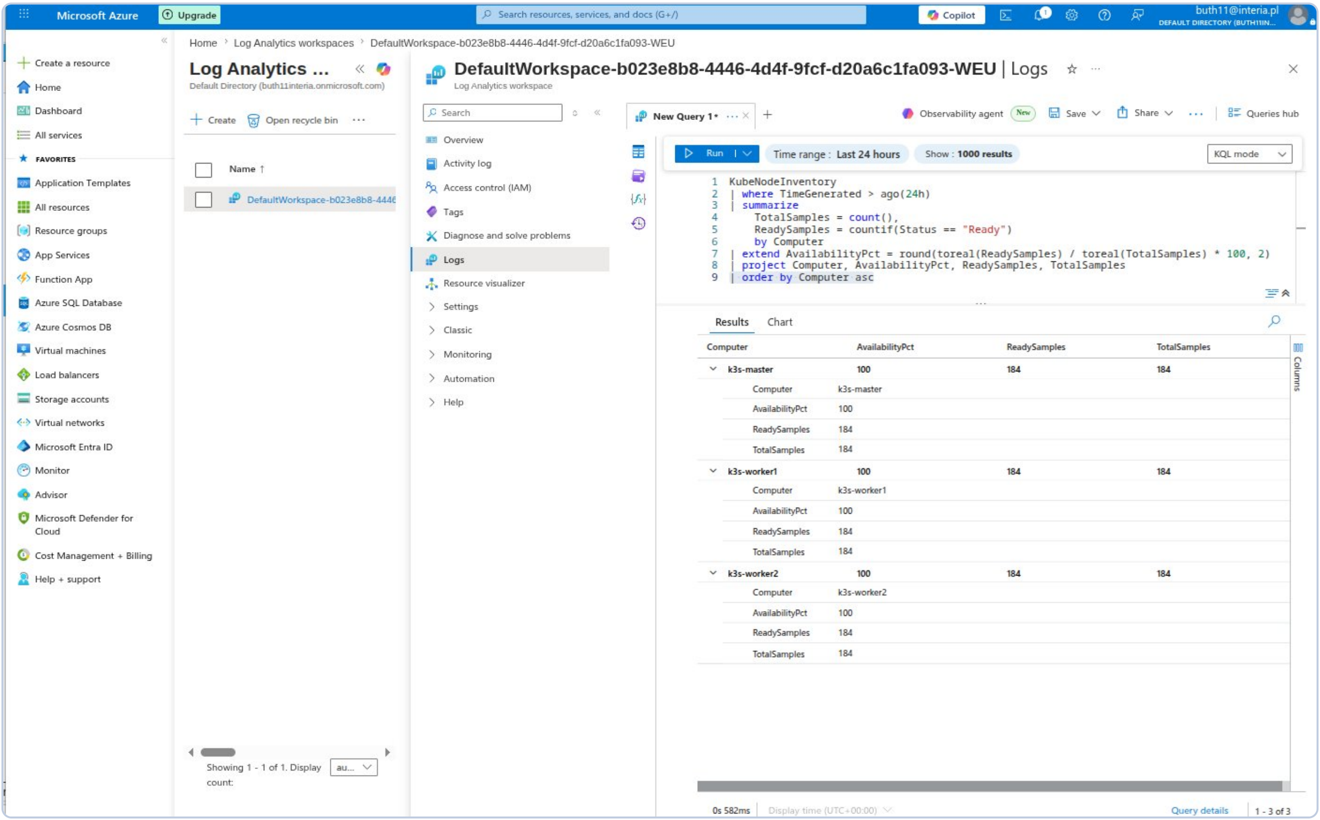
KQL-2: azure-arc 12 Running, default 3 Running, kube-system 14+2.



KQL-3: 13 events 24h — Rebooted, Unhealthy (temporary), FailedScheduling.



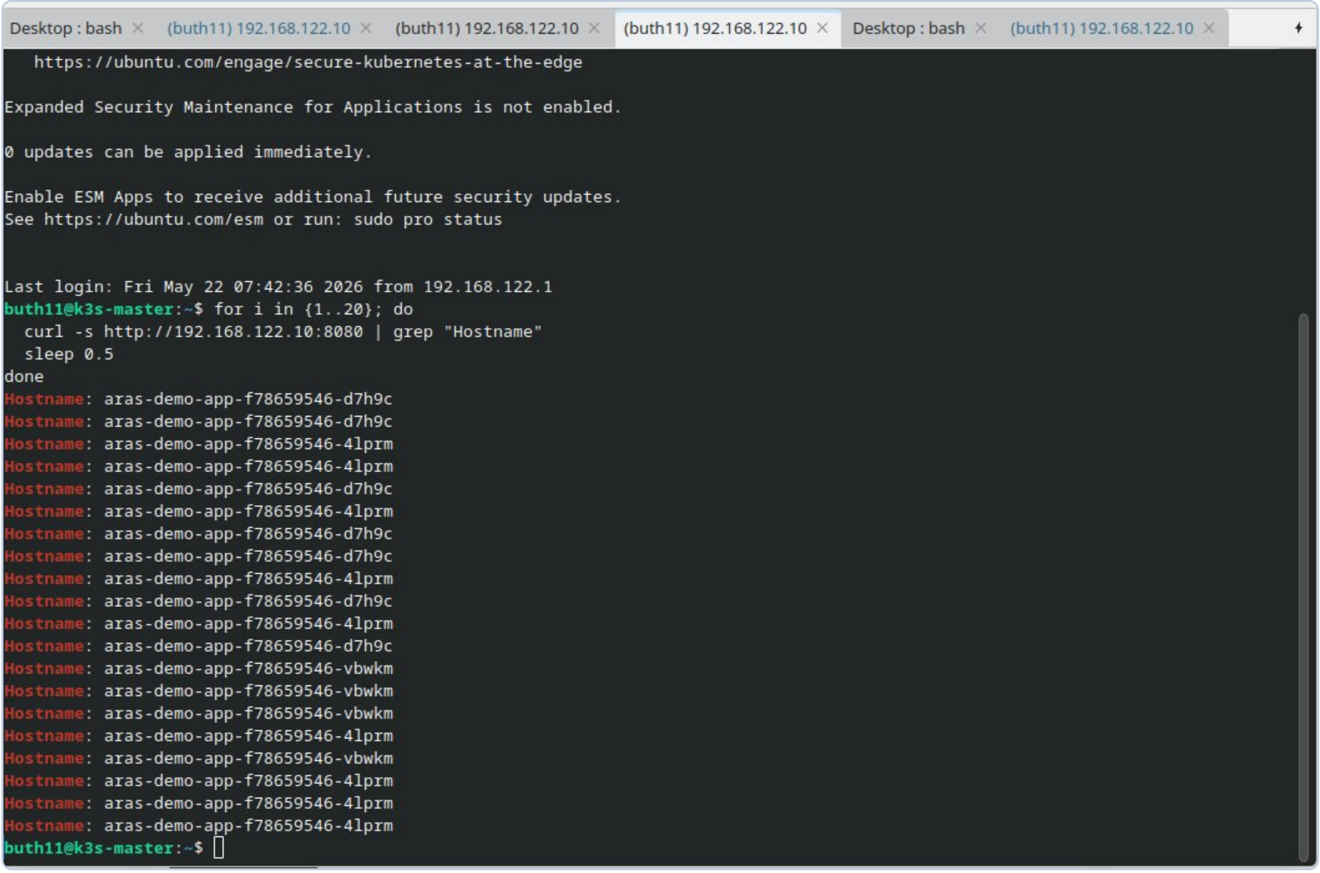
KQL-4: 27 pods with restarts after VM reboot — all Running.



KQL-5: SLO 100% — 184/184 samples Ready.

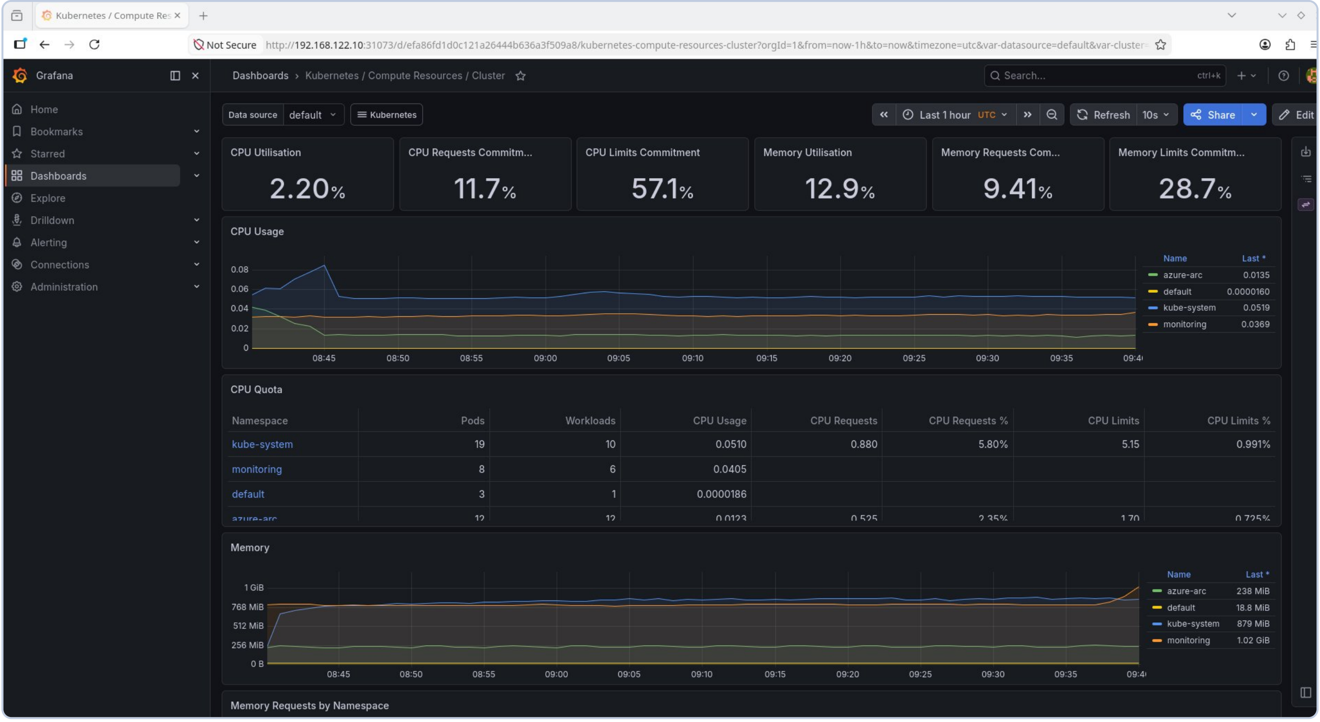
SLO 100% in 24h window. Error budget intact even after full VM restart.

Chapter 7: Load Balancing



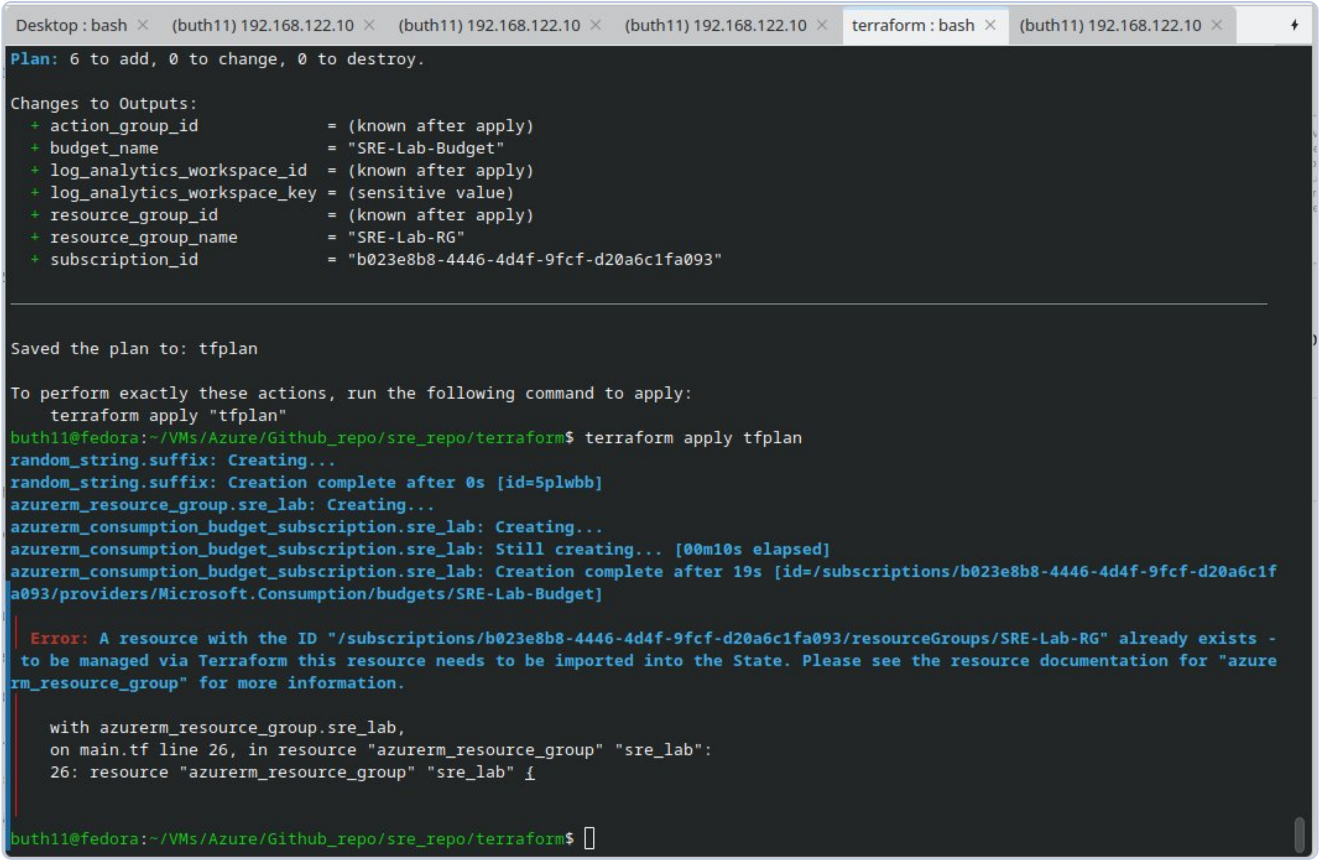
Screenshot 5: Round-robin — 20 requests across 3 pods, 900 parallel with zero errors.

Chapter 8: Prometheus & Grafana

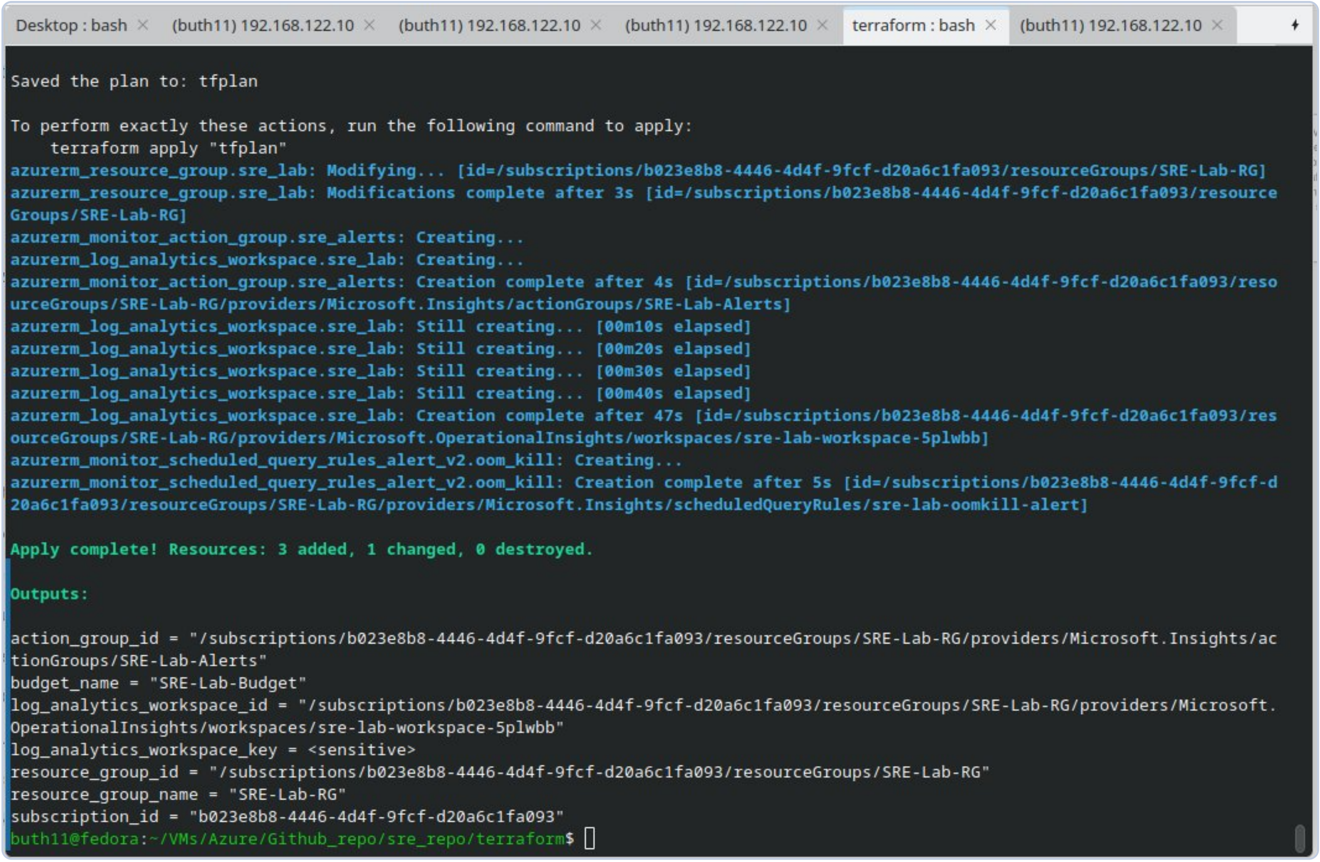


Screenshot 6: Grafana — CPU 2.20%, RAM 12.9%, CPU Quota per namespace.

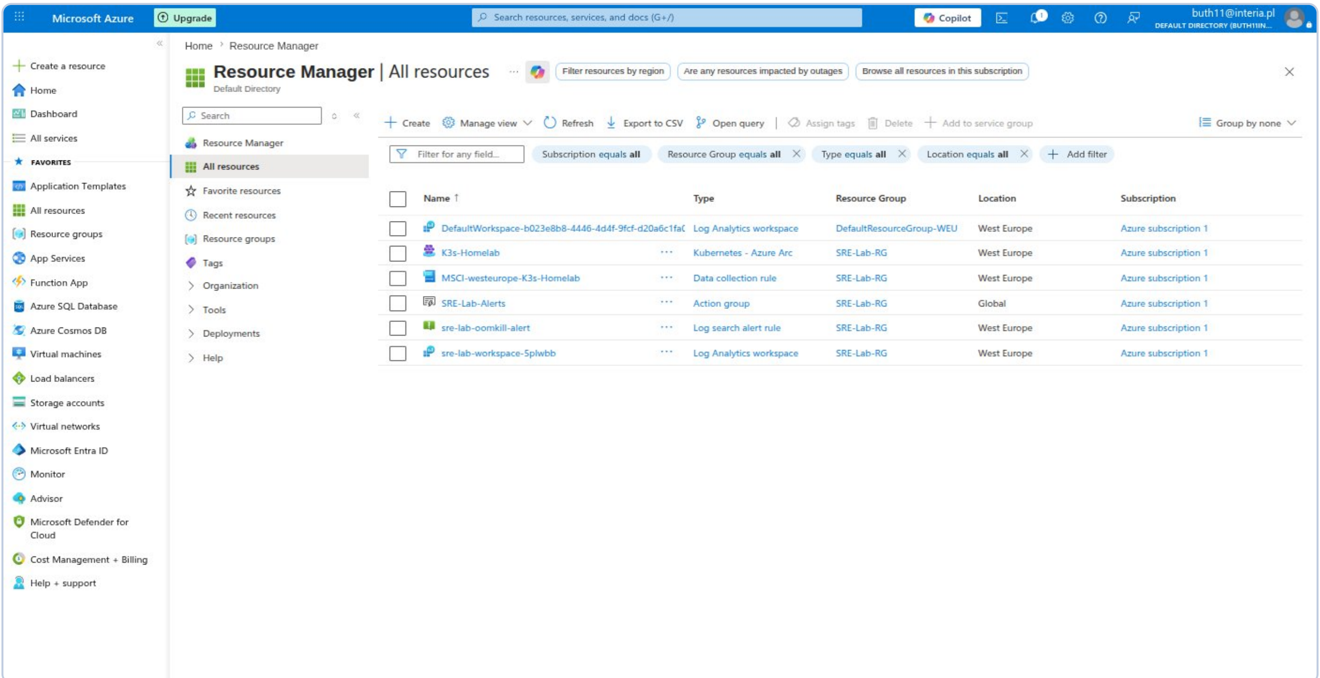
Chapter 9: Terraform IaC



Terraform Plan: 6 resources, 0 changes.

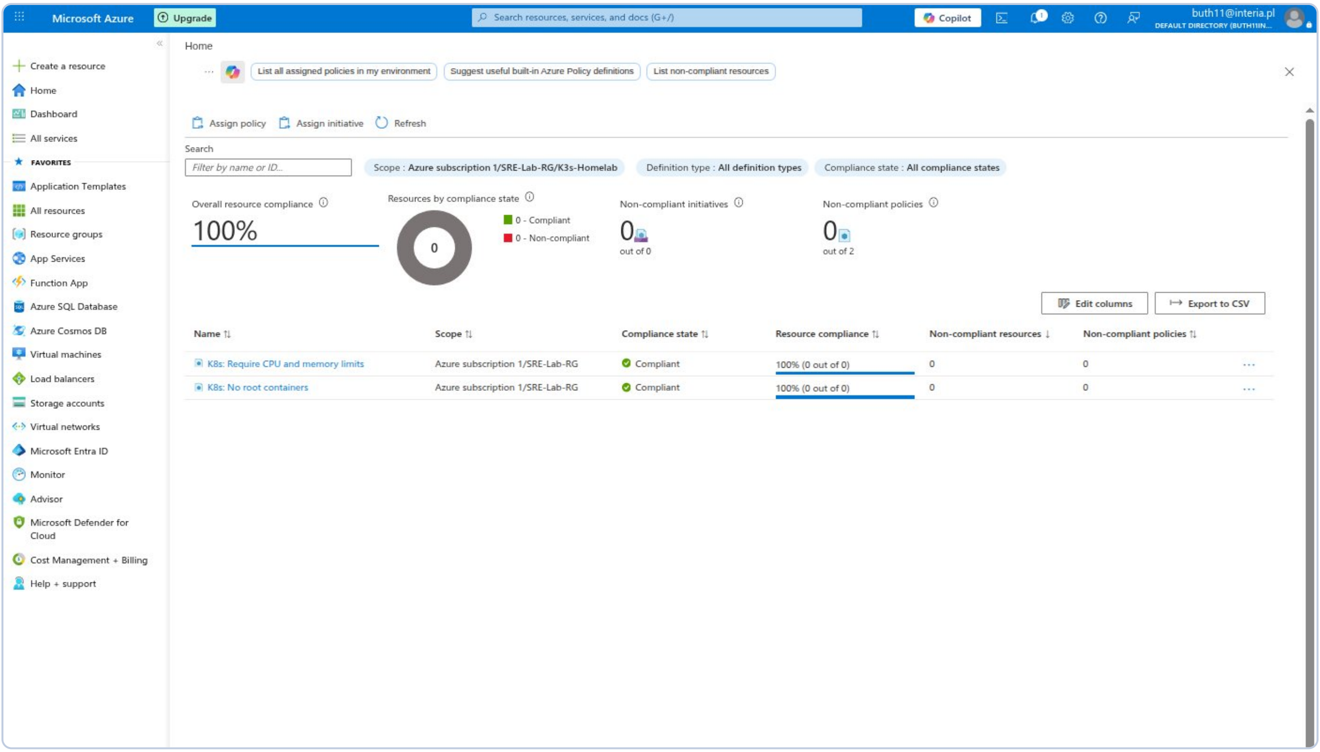


Terraform Apply: 3 added, 1 changed.



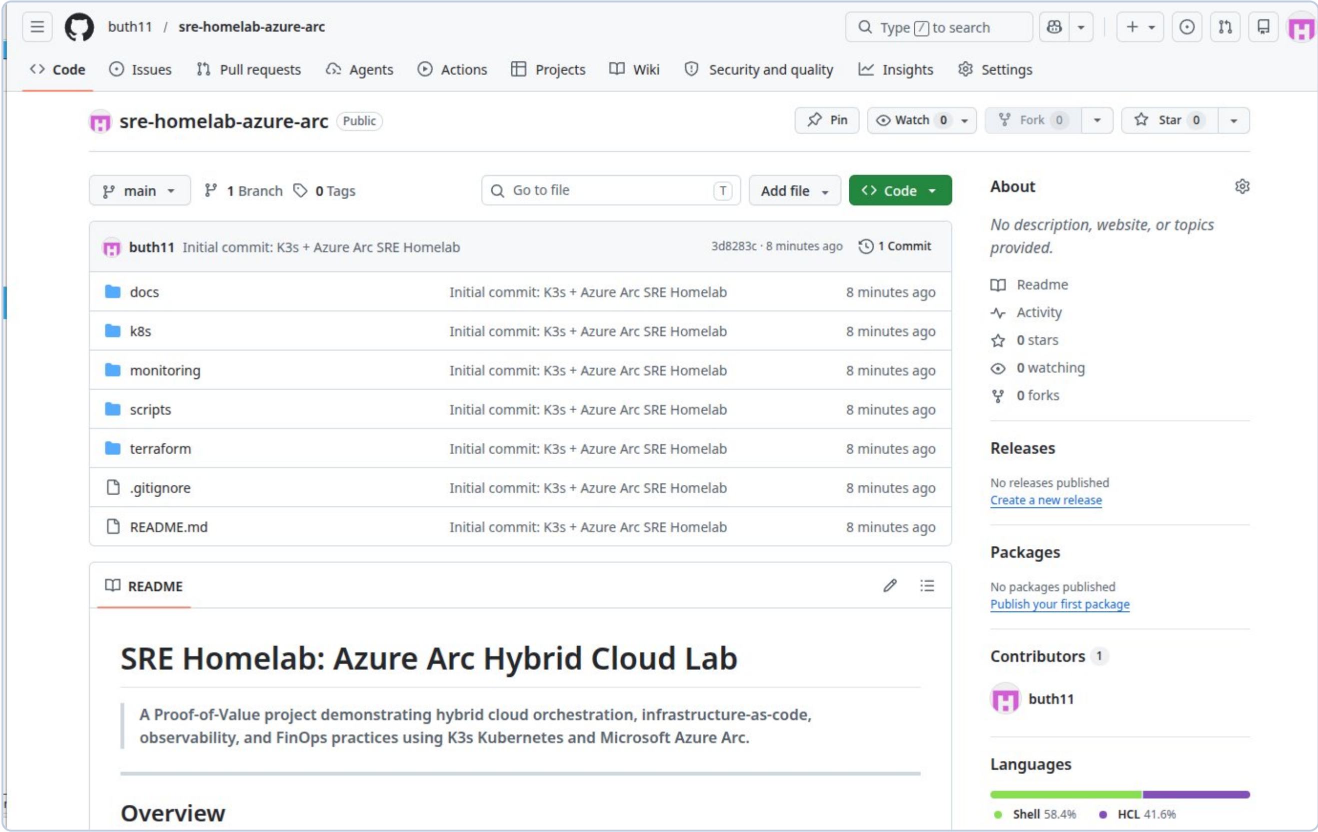
Azure Portal: Terraform resources in SRE-Lab-RG.

Chapter 10: Azure Policy



Azure Policy: No root containers + CPU/memory limits — both Compliant.

Chapter 11: GitHub



GitHub repo sre-homelab-azure-arc — public, Shell 58.4%, HCL 41.6%.

Chapter 12: Part 2 — Service Principal, Remote State, CI/CD

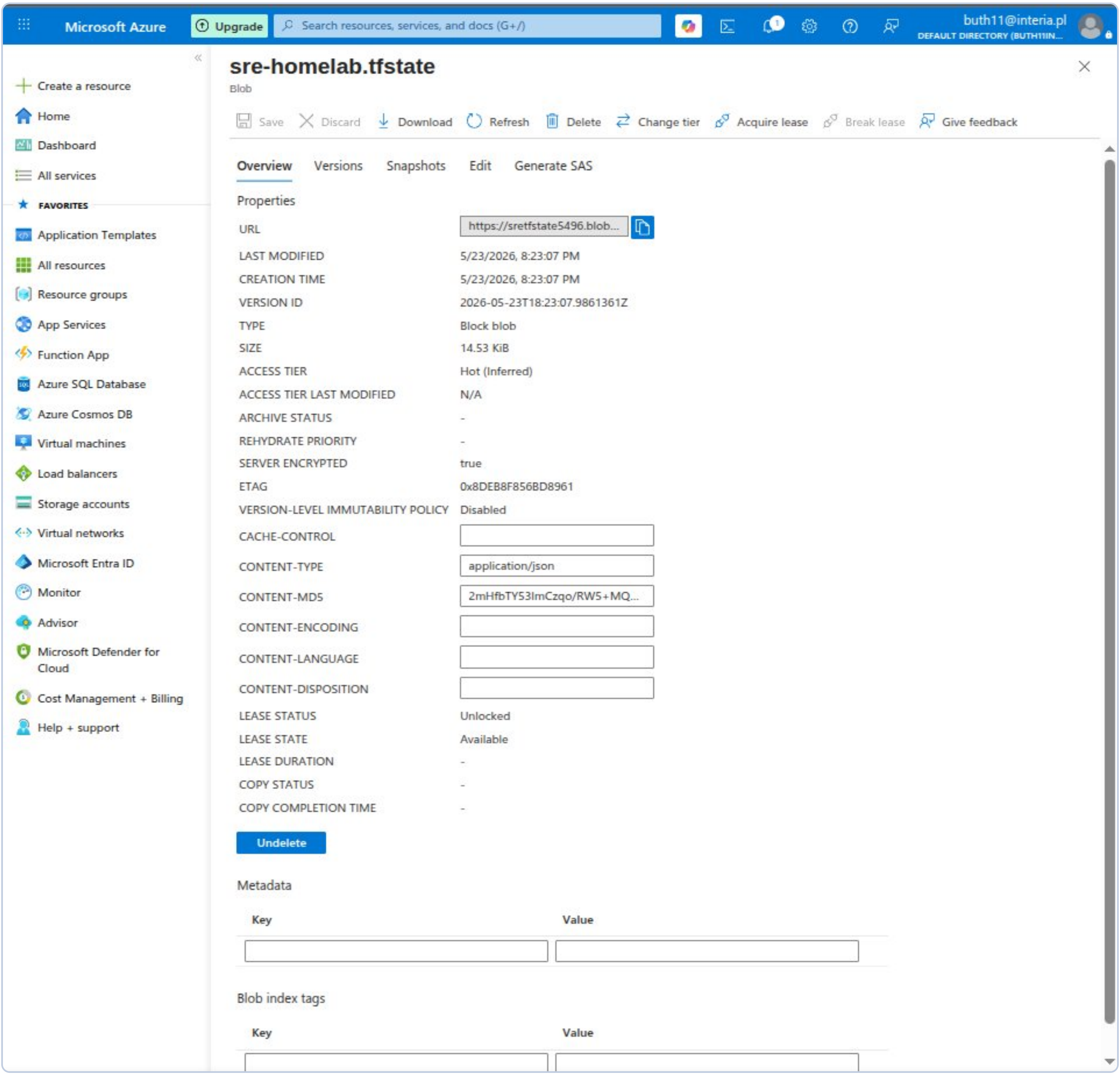
Remote State

File sre-homelab.tfstate — 14.53 KiB, server encrypted, versioning active. State lock visible: 'Acquiring state lock' / 'Releasing state lock' on every terraform plan.

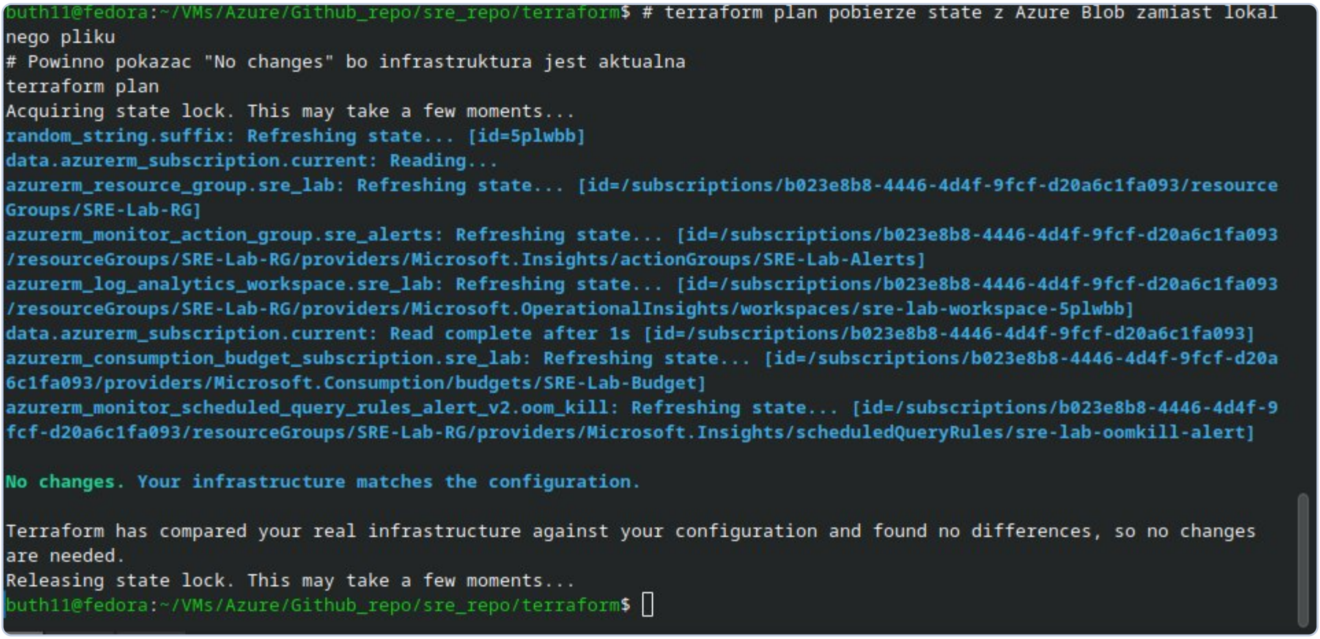
Azure DevOps CI/CD Pipeline

Variable Group terraform-credentials: ARM_CLIENT_ID, ARM_CLIENT_SECRET (hidden), ARM_SUBSCRIPTION_ID, ARM_TENANT_ID. Environment production: Approval Gate active.

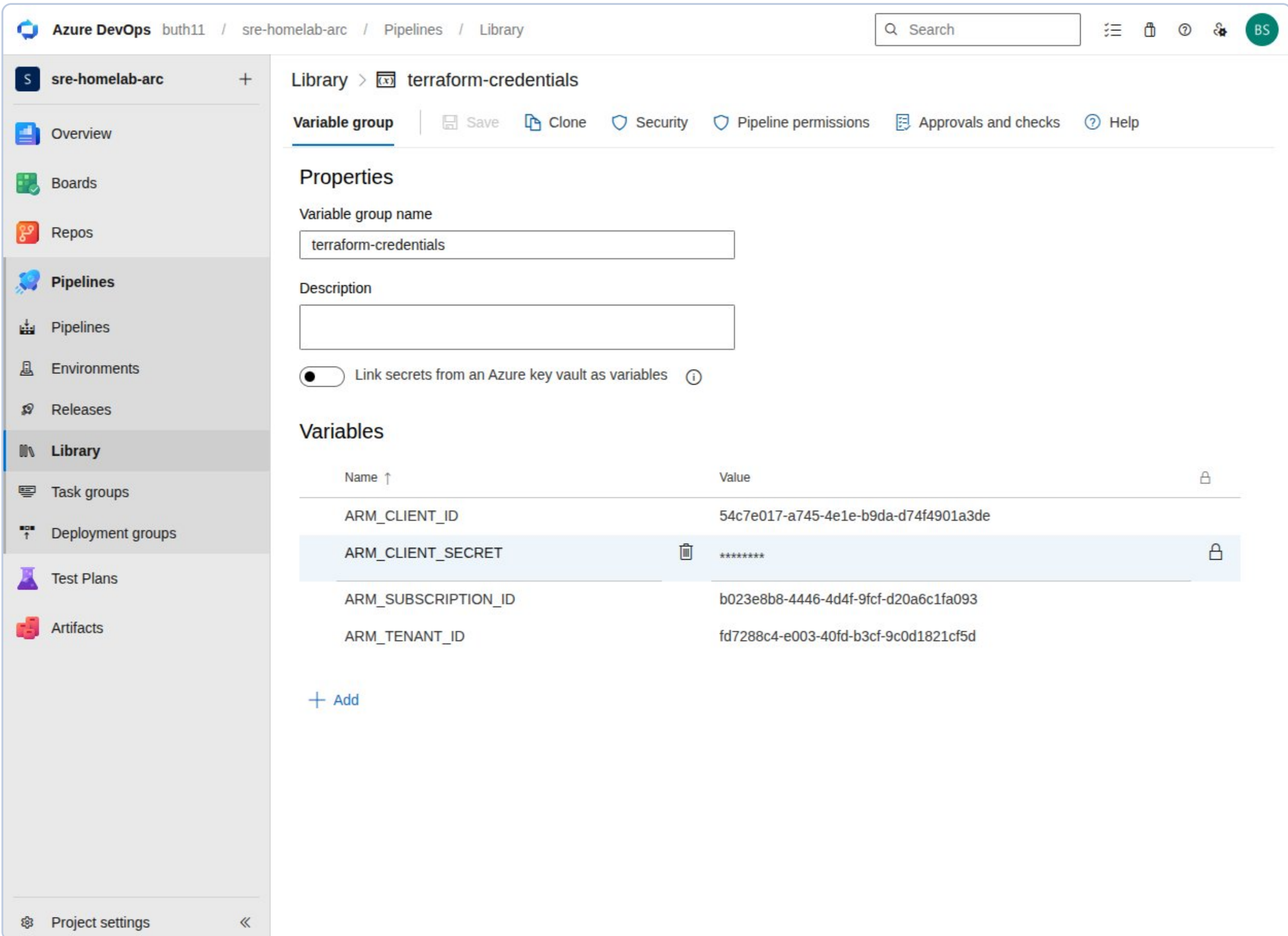
Result: Pipeline complete — both stages green, 4m 24s. Stage 1 (37s): validate + plan. Stage 2 (49s): apply after approval gate.



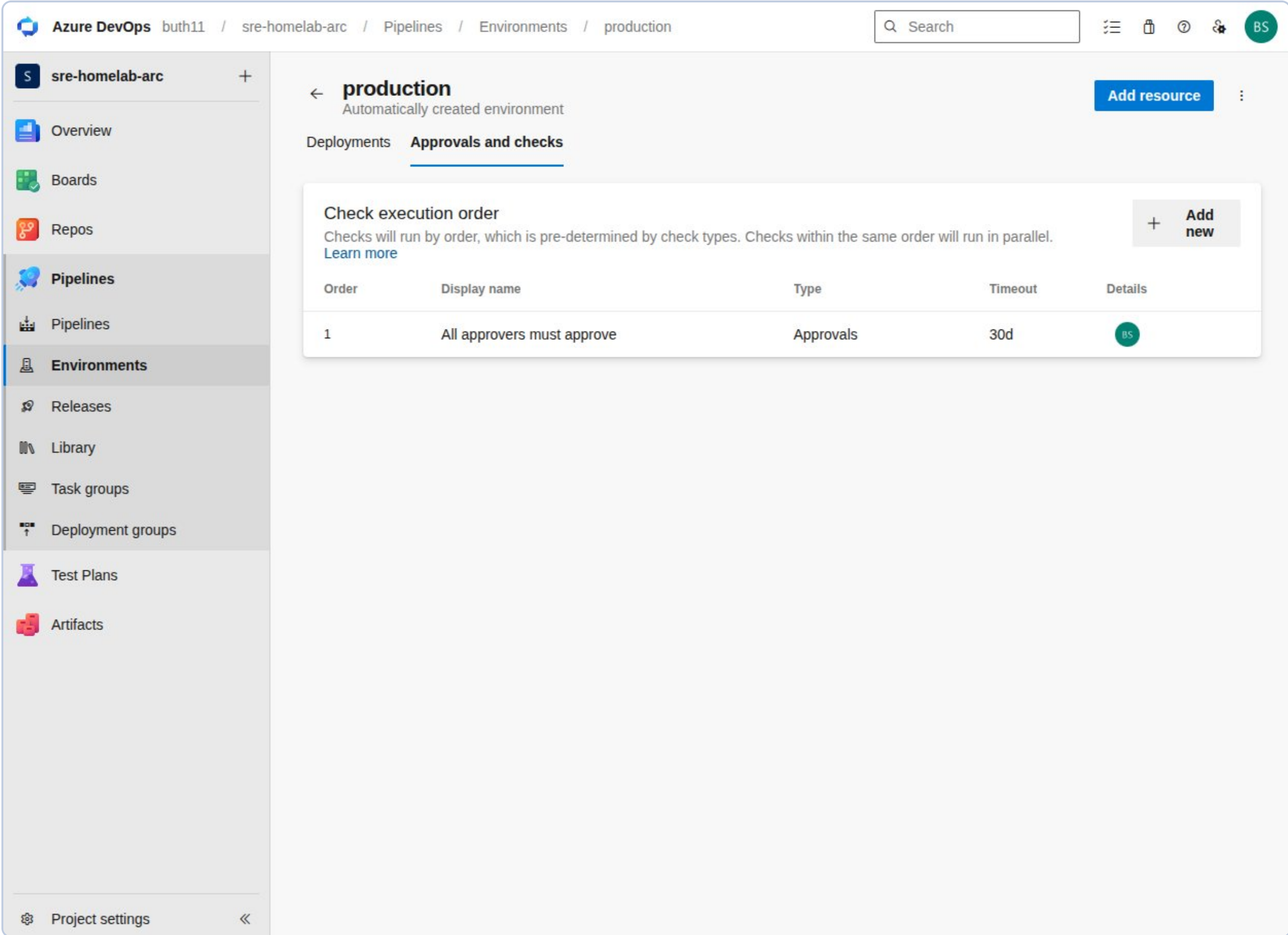
Screenshot W2-2: Azure Portal — blob properties: 14.53 KiB, encrypted, versioning active.



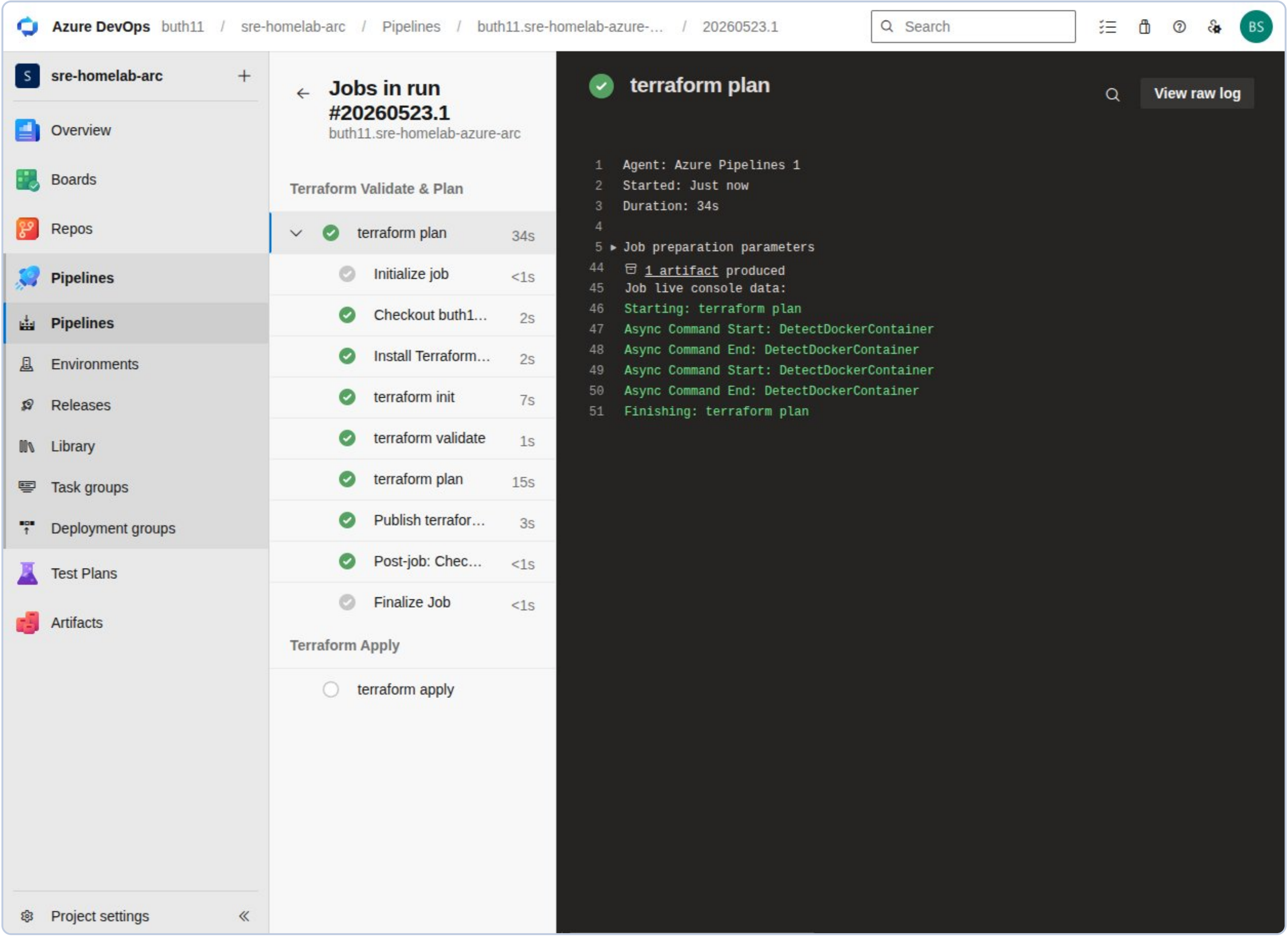
Screenshot W2-3: terraform plan — Acquiring/Releasing state lock. No changes.



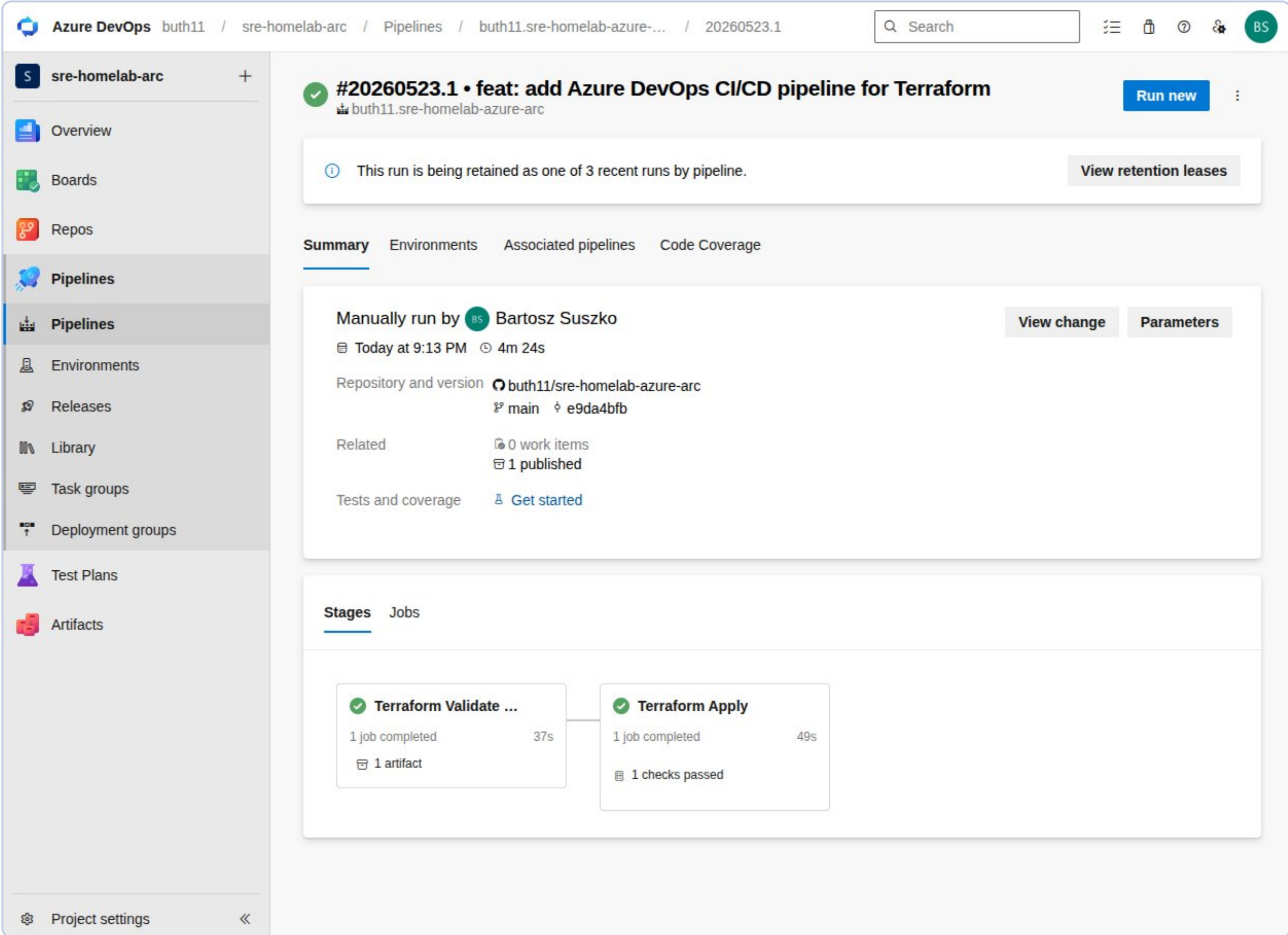
Screenshot W2-4: Azure DevOps Variable Group — ARM_* with hidden CLIENT_SECRET.



Screenshot W2-5: Environment production — Approval Gate active.



Screenshot W2-6: Stage 1 — all steps green, 1 artifact.



Screenshot W2-7: Pipeline complete — both stages green, 4m 24s.

Chapter 13: Part 3 — Advanced Kubernetes Operations

PART 3: Advanced Kubernetes Operations

Ingress with TLS — HTTPS routing by domain

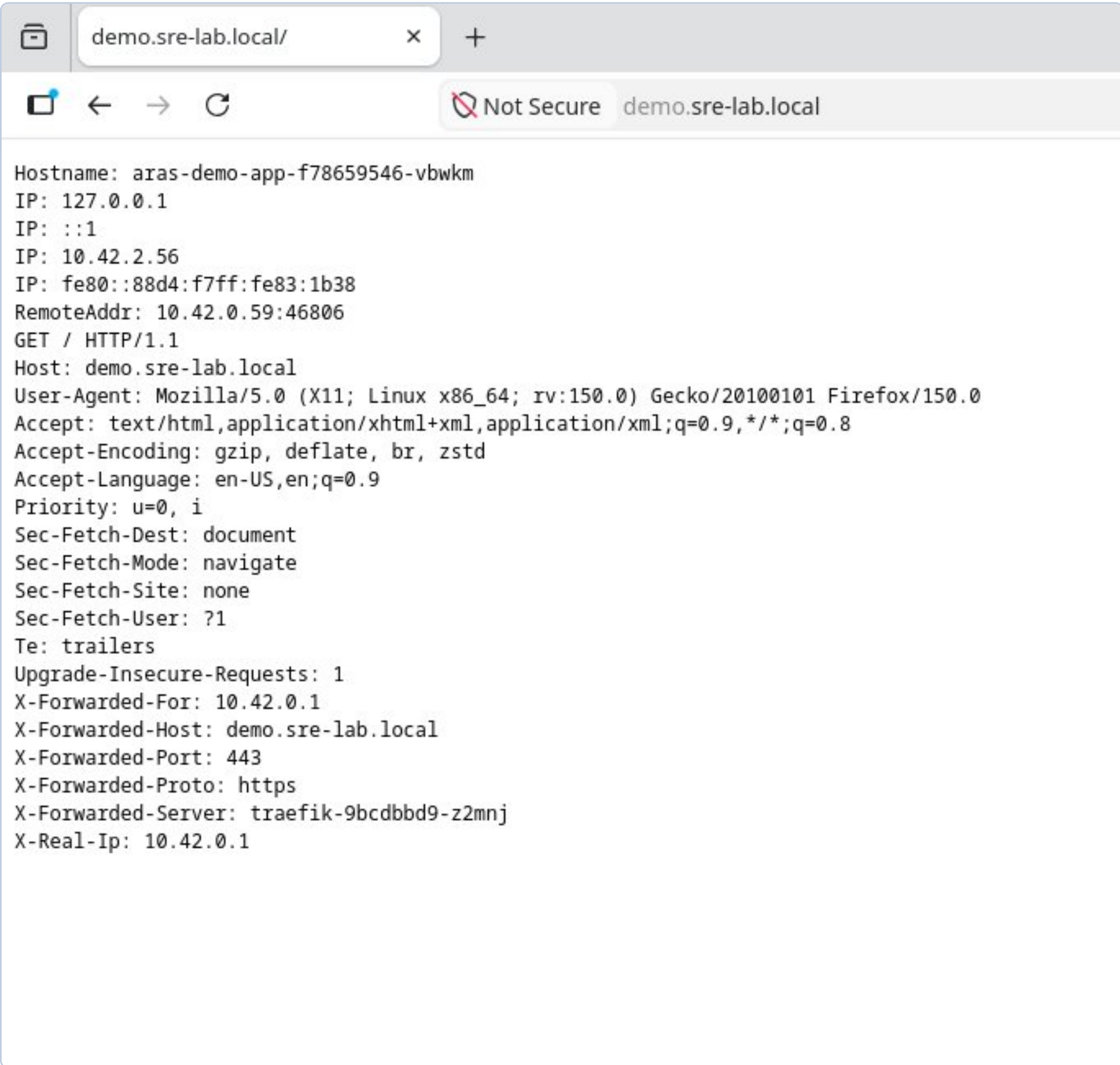
Traefik Ingress Controller configured for HTTPS. Self-signed TLS certificate with SAN, stored as Kubernetes Secret. Routing: demo.sre-lab.local → aras-demo-app:8080.

Certificate: openssl x509 self-signed, SAN for demo.sre-lab.local, valid 365 days

Secret: type kubernetes.io/tls in namespace default

Ingress: host-based routing through Traefik on port 443

Verification: X-Forwarded-Proto: https, X-Forwarded-Server: traefik in response



> Screenshot W3-1: Firefox browser — https://demo.sre-lab.local working. Visible: Pod hostname, X-Forwarded-Proto: https, X-Forwarded-Server: traefik-9bcdbbd9-z2mnj, X-Forwarded-Port: 443.

HPA — CPU-based pod autoscaling

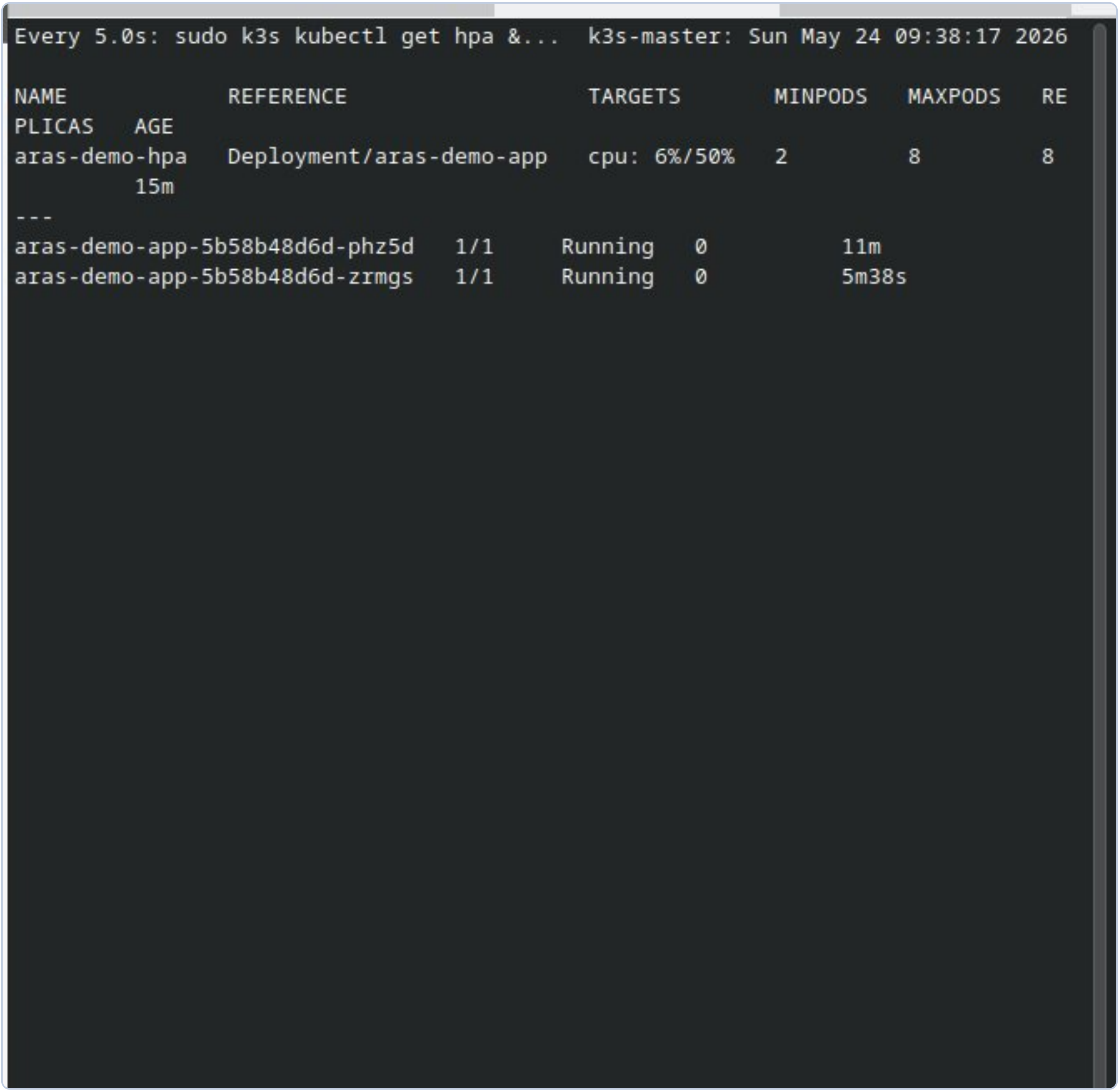
Horizontal Pod Autoscaler configured for the aras-demo-app Deployment. HPA queries metrics-server every 15 seconds for CPU usage. When average pod CPU exceeds 50% — it adds replicas. When it drops below 50% for 5 minutes — it removes pods.

- Range: minReplicas: 2, maxReplicas: 8, targetCPU: 50%
- Load test: ab -n 50000 -c 50 — 1621 req/s over 30 seconds
- Scale-up: CPU 860% → automatic scaling to 8 replicas
- Scale-down: CPU 6% after 5 minutes → return to 2 replicas (minimum)

```
Every 5.0s: sudo k3s kubectl get hpa &...  k3s-master: Sun May 24 09:32:49 2026

NAME            REFERENCE                  TARGETS         MINPODS    MAXPODS
REPLICAS   AGE
aras-demo-hpa   Deployment/aras-demo-app   cpu: 860%/50%   2          8
4          9m55s
---
aras-demo-app-5b58b48d6d-5vwl4   1/1     Running   0          24s
aras-demo-app-5b58b48d6d-7mbln   1/1     Running   0          10s
aras-demo-app-5b58b48d6d-859c7   1/1     Running   0          6m32s
aras-demo-app-5b58b48d6d-96qs4   1/1     Running   0          10s
aras-demo-app-5b58b48d6d-phz5d   1/1     Running   0          6m27s
aras-demo-app-5b58b48d6d-sbv4q   1/1     Running   0          10s
aras-demo-app-5b58b48d6d-xx94p   1/1     Running   0          24s
aras-demo-app-5b58b48d6d-zrmgs   1/1     Running   0          10s
```

Screenshot W3-2: HPA scale-up — cpu: 860%/50%, REPLICAS: 8 (max), 8 pods Running. Automatic scaling in action during ab load test.



Screenshot W3-3: HPA scale-down — cpu: 6%/50%, REPLICAS: 2 (min). After 5 minutes without traffic HPA automatically removed 6 unnecessary pods.

NetworkPolicy — intra-cluster firewall

By default all pods in Kubernetes can communicate with all pods. NetworkPolicy defines rules like a firewall — which namespaces and pods can connect to the demo application. Deployed a policy blocking all incoming traffic except from allowed sources.

- Allowed: kube-system (Traefik), monitoring (Prometheus), aras-demo-app replicas
- Blocked: everything else
- Verification: unauthorized pod — timeout/blocked. Traefik — application response

RBAC — least privilege for applications

Role-Based Access Control — every application should have its own account (ServiceAccount) with exactly the permissions it needs. Least privilege principle: if an application doesn't need to delete pods, it shouldn't be able to — even if compromised by an attacker.

- ServiceAccount: aras-demo-sa (instead of shared default)
- Role: get/list/watch on Pods and ConfigMaps — nothing more
- Verification: can-i list pods = yes, can-i delete pods = no

```
buth11@k3s-master:~$ # Test 1: nieautoryzowany Pod probuje polaczyc sie z aplik
acja
# timeout 5 = czekamy max 5 sekund na odpowiedz
# Oczekiwany wynik: brak odpowiedzi / timeout - polityka blokuje
echo "=== Test: nieautoryzowany dostep (powinien byc zablokowany) ==="
sudo k3s kubectl run test-blocked \
  --image=curlimages/curl \
  --rm -it \
  --restart=Never \
  -- curl -s --max-time 5 http://aras-demo-app:8080 || echo "ZABLOKOWANY - Netw
orkPolicy dziala"

# Test 2: sprawdzamy ze Ingress (Traefik) nadal dziala
# Traefik jest w kube-system wiec ma dostep
echo "=== Test: dostep przez Ingress (powinien dzialac) ==="
curl -sk https://demo.sre-lab.local | head -2
=== Test: nieautoryzowany dostep (powinien byc zablokowany) ===
pod "test-blocked" deleted from default namespace
pod default/test-blocked terminated (Error)
ZABLOKOWANY - NetworkPolicy dziala
=== Test: dostep przez Ingress (powinien dzialac) ===
buth11@k3s-master:~$ # Test dostępu przez Traefik - uzywamy IP zamiast domeny
# Traefik nasluchuje na porcie 80 i 443 na wszystkich wezlach
# -H podajemy naglowek Host zeby Traefik wiedzial do ktorego serwisu kierowac
echo "=== Test dostep przez Traefik (kube-system) ==="
curl -s -H "Host: demo.sre-lab.local" http://192.168.122.10 | head -3

# Dodatkowo sprawdzamy ze bezposredni dostep do portu 8080
# przez serwis nadal dziala (LoadBalancer Service jest poza NetworkPolicy)
echo "=== Test dostep przez LoadBalancer Service ==="
curl -s http://192.168.122.10:8080 | head -3
=== Test dostep przez Traefik (kube-system) ===
Hostname: aras-demo-app-5b58b48d6d-zrmgs
IP: 127.0.0.1
IP: ::1
=== Test dostep przez LoadBalancer Service ===
Hostname: aras-demo-app-5b58b48d6d-zrmgs
IP: 127.0.0.1
IP: ::1
buth11@k3s-master:~$
```

Screenshot W3-4: RBAC verification — ServiceAccount aras-demo-sa: list pods=yes (allowed), delete pods=no (blocked). Least privilege principle confirmed.

Chapter 14: Postmortem POST-001

Field	Value
Severity	P2
Duration	Under 7 minutes
Root Cause	Typo: ks3-worker1 instead of k3s-worker1
Status	Resolved — regex validation in scripts

Chapter 15: Results and Conclusions

Metric	Part 1	Part 2	Part 3
Nodes	3/3 Ready	—	—
SLO	100%	—	—
Cost	$0.00	$0.00	$0.00
Terraform state	Local	Remote (Blob)	—
CI/CD	—	Azure DevOps 4m24s	—
Ingress TLS	—	—	HTTPS verified
HPA	—	—	3 to 8 to 2 replicas
NetworkPolicy	—	—	Unauthorized blocked
RBAC	—	—	Least privilege
Issues	6	8	10

Chapter 16: Troubleshooting Log

ISSUE-001 KQL: CPUCapacityNanoCores column missing

```
project operator: Failed to resolve scalar expression named CPUCapacityNanoCores
```

Resolution: KubeNodeInventory | getschema — simplified query to available columns.

ISSUE-002 Azure Policy: MissingPolicyParameter cpuLimit/memoryLimit

```
(MissingPolicyParameter) policy assignment k8s-resource-limits is missing cpuLimit, memoryLimit
```

Resolution: Added --params with cpuLimit: 500m and memoryLimit: 512Mi.

ISSUE-003 Terraform: Resource Group already exists (brownfield)

```
Error: A resource with the ID .../SRE-Lab-RG already exists
```

Resolution: terraform import azurerm_resource_group.sre_lab /subscriptions/.../SRE-Lab-RG

ISSUE-004 Terraform: Saved plan is stale after import

```
Error: Saved plan is stale — state was changed after plan was created
```

Resolution: New terraform plan -out=tfplan after import.

ISSUE-005 Container Insights missing after VM restart

Resolution: az k8s-extension create --name azuremonitor-containers — agents returned in 37s.

ISSUE-006 POST-001: etcd Split-Brain (ks3 vs k3s)

```
FATA[0034] Node password rejected, duplicate hostname
```

Resolution: kubectl delete node + rm /etc/rancher/node/ + restart k3s-agent. Scripts updated with regex validation.

ISSUE-007 Azure DevOps: TerraformInstaller task missing

```
A task is missing: TerraformInstaller
```

Resolution: Install from Marketplace: ms-devlabs.custom-terraform-tasks.

ISSUE-008 Pipeline: permission for Variable Group

```
This pipeline needs permission to access a resource
```

Resolution: View → Permit — one-time authorization.

ISSUE-009 sudo does not expand ~ in paths

```
error: Cannot read file ~/ingress/server.crt, open ~/ingress/server.crt: no such file or directory
```

Root Cause: sudo runs the command as root which has a different home directory. Tilde ~ expands to /root/ instead of /home/buth11/.

Resolution: Use full path: /home/buth11/ingress/server.crt instead of ~/ingress/server.crt

ISSUE-010 HPA shows cpu: unknown — missing resource requests

```
NAME TARGETS MINPODS MAXPODS REPLICAS — cpu: <unknown>/50%
```

Root Cause: HPA calculates CPU percentage as current_usage/requests*100. Without defined requests in the Deployment, HPA has no baseline to calculate percentages from.

Resolution: Added resources.requests to Deployment: cpu: 10m, memory: 32Mi. After rolling update HPA started showing correct values: cpu: 0%/50%.

PART 4
GitOps, Security Scanning & Runtime Protection

This section covers the final part of the SRE Homelab project:

- FluxCD 2.8.8 — GitOps controller: automatic cluster reconciliation from Git
- Trivy 0.70.0 — Container image CVE scanning: 1 CRITICAL + 12 HIGH found
- Defender for Cloud — Runtime security DaemonSet on all 3 nodes

Chapter 14: FluxCD — GitOps for Kubernetes

FluxCD bootstrapped from GitHub repository via SSH key (github_sre). Four controllers installed in flux-system namespace — all healthy.

Bootstrap command:

```
flux bootstrap git \ --url=ssh://git@github.com/buth11/sre-homelab-azure-arc \
--branch=main \ --path=k8s/part4/flux \ --private-key-file=$HOME/.ssh/github_sre
```

Verification — all components healthy:

```
flux check --pre ■ checking prerequisites ✔ Kubernetes 1.35.4+k3s1 >=1.33.0-0 ✔
prerequisites checks passed flux get all NAME REVISION SUSPENDED READY MESSAGE
gitrepository/flux-system main@sha1:d169d6a0 False True stored artifact
kustomization/flux-system main@sha1:952c1ea0 False True Applied revision
kustomization/gitops-demo main@sha1:952c1ea0 False True Applied revision kubectl get pods
-n flux-system helm-controller-5984cc88cf-94jkv 1/1 Running 0 117s
kustomize-controller-d6bb746df-9khwf 1/1 Running 0 117s
notification-controller-7bc5c97f8d-jdflb 1/1 Running 0 117s
source-controller-745c67ff9-2g8qd 1/1 Running 0 117s
```

GitOps test — scaling via git push:

Changed replicas from 2 to 3 in deployment.yaml, committed and pushed. Flux automatically applied the change — third pod appeared without any kubectl apply.

```
# Before push: 2 pods kubectl get pods -n gitops-demo gitops-app-5b9747bdcb-2rqw9 1/1
Running 0 11m gitops-app-5b9747bdcb-bjvtd 1/1 Running 0 11m # After: git push with
replicas: 3 gitops-app-5b9747bdcb-2rqw9 1/1 Running 0 12m gitops-app-5b9747bdcb-bjvtd 1/1
Running 0 12m gitops-app-5b9747bdcb-gttmd 1/1 Running 0 93s ← added by Flux
```

Chapter 15: Trivy — Container Image CVE Scanning

Trivy 0.70.0 installed and used to scan traefik/whoami:latest. All 40 vulnerabilities are in Go stdlib v1.24.1 — the image uses an outdated Go version.

Scan results summary:

Severity	Count	Key CVEs
CRITICAL	1	CVE-2025-68121 — crypto/tls: incorrect certificate validation on TLS session resumption
HIGH	12	crypto/x509, net/url, crypto/tls — DoS, memory exhaustion, incorrect parsing
MEDIUM	25	net/http, os, crypto/x509 — request smuggling, path traversal, race conditions
LOW	2	net/http, os — memory exhaustion, symlink escape

Key CRITICAL vulnerability:

```
Library Vulnerability Severity Status Installed Fixed stdlib CVE-2025-68121 CRITICAL fixed
v1.24.1 1.24.13, 1.25.7 crypto/tls: Incorrect certificate validation during TLS session
resumption https://avd.aquasec.com/nvd/cve-2025-68121
```

Scan reports committed to repository:

- k8s/part4/trivy/whoami-scan.json — full report (JSON format)
- k8s/part4/trivy/whoami-scan.txt — CRITICAL+HIGH only (table format)

Fix: update image to version compiled with Go v1.24.13+. In production, CI pipeline blocks deploy of images with CRITICAL CVEs.

Chapter 16: Defender for Cloud — Runtime Security

Microsoft Defender for Containers enabled at Standard tier (30-day free trial). Extension installed on Arc cluster — provisioningState: Succeeded.

Installation:

```
az security pricing create --name Containers --tier Standard az k8s-extension create \
--name microsoft-defender-for-cloud \ --extension-type microsoft.azuredefender.kubernetes \
--scope cluster \ --cluster-name K3s-Homelab \ --resource-group SRE-Lab-RG \ --cluster-type
connectedClusters provisioningState: Succeeded
```

Defender pods running in mdc namespace:

```
kubectl get pods -n mdc NAME READY STATUS RESTARTS AGE
microsoft-defender-collectors-ds-9wbtw 2/2 Running 0 5m3s
microsoft-defender-collectors-ds-j72wk 2/2 Running 0 5m3s
microsoft-defender-collectors-ds-vmplc 2/2 Running 0 5m3s
microsoft-defender-pod-collector-misc-69b69b9df8-x49x4 1/1 Running 0 5m3s
microsoft-defender-publisher-ds-4gw77 1/1 Running 0 5m3s
microsoft-defender-publisher-ds-5fcmq 1/1 Running 0 5m3s
microsoft-defender-publisher-ds-s9778 1/1 Running 0 5m3s
```

DaemonSet covers all 3 nodes — master, worker1, worker2. Collectors gather security metrics per node, Publisher sends them to Azure Security Center.

Part 4 Results Summary

Metric	Part 1	Part 2	Part 3	Part 4
Nodes	3/3 Ready	—	—	—
SLO	100%	—	—	—
Cost	$0.00	$0.00	$0.00	$0.00
Terraform state	Local	Remote (Blob)	—	—
CI/CD	—	DevOps 4m24s	—	—
Ingress TLS	—	—	HTTPS verified	—
HPA	—	—	3→8→2 replicas	—
NetworkPolicy	—	—	Blocked	—
RBAC	—	—	Least privilege	—
GitOps	—	—	—	FluxCD deployed
Image scanning	—	—	—	1 CRITICAL, 12 HIGH
Runtime security	—	—	—	Defender (3 nodes)
Issues documented	6	8	10	11

PART 5
cert-manager — Automated TLS Certificate Management

cert-manager v1.20.2 installed via Helm. Automatic TLS certificate lifecycle management — issuance, renewal, and rotation without manual intervention.

- cert-manager v1.20.2 — installed via Helm (jetstack/cert-manager)
- ClusterIssuer selfsigned-issuer — bootstrap CA
- ClusterIssuer sre-lab-ca-issuer — local CA signing certificates
- Automatic cert issuance in 22 seconds via Ingress annotation
- Auto-renewal verified: renewBefore=6m before expiry

Chapter 17: cert-manager — Automated TLS

Installation via Helm:

```
helm repo add jetstack https://charts.jetstack.io helm install cert-manager
jetstack/cert-manager \ --namespace cert-manager \ --create-namespace \ --set
crds.enabled=true kubectl get pods -n cert-manager cert-manager-5957746d66-kdkk5 1/1
Running 0 cert-manager-cainjector-567c6b47ff-c5x2b 1/1 Running 0
cert-manager-webhook-7cc5c588cb-zm8d4 1/1 Running 0
```

ClusterIssuers — both Ready:

```
kubectl get clusterissuer NAME READY AGE selfsigned-issuer True 32s sre-lab-ca-issuer True
32s kubectl get certificate -n cert-manager NAME READY SECRET AGE sre-lab-ca True
sre-lab-ca-secret 32s
```

Ingress annotation — certificate issued automatically in 22s:

```
# Ingress annotation only: annotations: cert-manager.io/cluster-issuer: sre-lab-ca-issuer #
cert-manager created the certificate automatically: kubectl describe certificate
demo-sre-lab-tls -n default Issuer: CN=sre-lab-ca Not After: 2026-08-25T09:09:34Z (90 days)
Renewal Time: 2026-07-26T09:09:34Z (auto-renew 30d before expiry) Status: Ready: True
Message: Certificate is up to date and has not expired # Verify TLS works: curl -sk
https://demo.sre-lab.local | grep Hostname Hostname: aras-demo-app-5b58b48d6d-phz5d openssl
x509 verification: Issuer: CN=sre-lab-ca Not After: Aug 25 09:09:34 2026 GMT
```

Part 3 vs Part 5 comparison:

Aspect	Part 3 — manual	Part 5 — cert-manager
Generation	openssl req -x509 ...	Automatic by controller
Secret	kubectl create secret tls	Created automatically
Renewal	Manual before expiry	Automatic (renewBefore)
Issuer	CN=demo.sre-lab.local	CN=sre-lab-ca (our CA)
Production	Not suitable	Let's Encrypt ClusterIssuer

Part 5 Results Summary

Metric	Part 1	Part 2	Part 3	Part 4	Part 5
Nodes	3/3 Ready	—	—	—	—
SLO	100%	—	—	—	—
Cost	$0.00	$0.00	$0.00	$0.00	$0.00
CI/CD	—	DevOps 4m24s	—	—	—
Ingress TLS	—	—	Manual self-signed	—	Auto cert-manager
HPA	—	—	3→8→2	—	—
GitOps	—	—	—	FluxCD	—
Image scan	—	—	—	1 CRIT+12 HIGH	—
TLS mgmt	—	—	Manual openssl	—	cert-manager auto
Issues	6	8	10	11	12